Zero-Identity Secure Messaging

A privacy-first messaging backend where users connect via codes, messages are encrypted at rest, and data automatically disappears.

Enter the secure space

WHAT IS ZISM

Zero-Identity Platform

A backend messaging system built for privacy. No emails, no phone numbers — just a random code and a password. Messages are encrypted in storage and wipe themselves after a set time.

0	6–8	24h
PII REQUIRED	CHAR USER CODE	MAX MESSAGE TTL

Core Goals

- Build a real-world backend system from scratch
- Implement secure authentication with hashed passwords
- Apply practical encryption — no overkill, no theatre
- Give users privacy and control over their data
- Keep the system simple, scalable, and actually buildable

Tech Stack

LAYER	TECHNOLOGY	NOTES
Frontend	Web UI	Vanilla HTML/CSS/JS or React
Backend	`FastAPI`	Python, async-ready
Database	`SQLite → PostgreSQL`	SQLAlchemy ORM
Auth	`JWT + bcrypt`	PyJWT + passlib
Encryption	`PyNaCl`	Symmetric, per-channel keys

This is a hobby project. The encryption is server-side — designed for learning and portfolio purposes. It's "encrypted at rest", not end-to-end. That's an honest, intentional tradeoff.

HONEST DESIGN

Security Model

The security model is upfront about what it can and cannot do. Honesty in design is a feature, not a flaw.

✦ GUARANTEED

- ✓ Messages encrypted in database
- ✓ Passwords hashed with bcrypt
- ✓ No plaintext message storage ever
- ✓ HTTPS network transport enforced

✦ NOT GUARANTEED

- ✗ Preventing screenshots
- ✗ Stopping message copies
- ✗ Full anonymity vs advanced tracking
- ✗ True end-to-end encryption

Security Principles

- Password protection — bcrypt hashing, plaintext never stored at any point
- Network security — HTTPS only, no interception vectors in transit
- Server trust model — validate all input server-side, trust nothing from the frontend
- Data hygiene — secrets live only in the backend, never in JavaScript

Browser Inspection Reality

USERS CAN SEE

- ! HTML structure
- ! CSS styling
- ! JavaScript logic

USERS CANNOT SEE

- ✓ Backend source code
- ✓ Database contents
- ✓ Server logic & secrets

Golden rule: `Never put secrets in the frontend.` API keys, encryption keys, database credentials — they belong on the server only.

IDENTITY SYSTEM

Auth
Without Identity

No email. No phone. Users are identified only by a randomly generated code. Authentication is code + password, with JWT tokens for session management.

Registration Flow

1. User opens the app for the first time
2. Backend generates a unique `user_code` — e.g. `X7K9P2` (6–8 chars, alphanumeric)
3. User sets their password — hashed with `bcrypt` before any storage

Users Table Schema

FIELD	TYPE	DESCRIPTION
`id`	`UUID`	Internal primary key
`user_code`	`String`	Public login identifier (e.g. X7K9P2)
`password_hash`	`String`	bcrypt-hashed password, never plaintext
`created_at`	`DateTime`	Account creation timestamp

Login Flow

1. User submits `user_code` + password
2. Backend finds user, verifies password via bcrypt comparison
3. On success, returns a signed `JWT token` for session management

Session Management

- → JWT sent in `Authorization: Bearer <token>` header on every request
- → Backend validates token on each protected route via `get_current_user` dependency
- → Stateless — no server-side session storage required

→ Token stored in `localStorage` client-side for session persistence on reload

CORE FEATURE

Messaging & Encryption

Friend / Channel System

1. User enters a friend's `user_code`
2. Backend verifies code exists and isn't your own, generates a PyNaCl `shared_secret`, creates a channel

Channels Schema

FIELD	TYPE
id	UUID
user1_id	UUID
user2_id	UUID
shared_secret	Bytes
created_at	DateTime

Messages Schema

FIELD	TYPE
id	UUID
channel_id	UUID
encrypted_text	Text
created_at	DateTime
expiry_time	DateTime

Encryption Design — PyNaCl

- → Each channel generates a unique `shared_secret` stored in the Channel row
- → Messages encrypted using PyNaCl `SecretBox` **before** writing to database
- → Decrypted only on authenticated fetch — plaintext never persisted
- → Simple symmetric encryption — honest, practical, good for learning

Message Expiry (TTL)

TTL OPTIONS

- 1 hour
- 4 hours
- 24 hours

CLEANUP LOGIC

- Store `expiry_time` per message
- Background thread on startup
- Hard-deletes expired rows

ENDPOINTS

API Reference

POST	/register	Create user, receive user_code
POST	/login	Authenticate, receive JWT token
POST	/channels/add-friend	Add friend by code, create channel
GET	/channels/	List user's active channels
POST	/messages/{channel_id}	Send encrypted message
GET	/messages/{channel_id}	Fetch & decrypt messages

BUILD PLAN

Dev Roadmap

PHASE 01
Foundation
Setup FastAPI, implement /register and /login endpoints

PHASE 02
Social Layer
Friend system, channel creation via user codes

PHASE 03
Messaging Core
Send and receive messages within channels

PHASE 04
Encryption
PyNaCl SecretBox — encrypt before save, decrypt on fetch

PHASE 05
Expiry System
TTL per message, background cleanup thread

PHASE 06
Polish & Ship
Testing, docs, GitHub push

Optional Future Features

Read receipts + auto-delete | Code rotation system | No-history mode | Rate limiting | WebSocket real-time | UI improvements

ZISM v1.0 · 2025 · samikhan.in · Absolute Privacy. Ephemeral Communication.